Review of C++

outline

- Difference between Python and C++
- The first program “Hello, world” (Sections 1.5, 1.6)
- Input and output
- Variable (Section 2.1)
- Assignment operator
- Condition
- Branch statements
  - if-statement
  - if-else statement
  - Nested if-else statement

Difference between python and C++

- Python is dynamically typed, whereas C++ is strongly typed.
  - Example in Python: n = 5 (correct syntax)
  - Example in C++: int n = 5; (correct syntax)
- Every statement in C++ ends with a semicolon (;)
- To enclose multiple statements in conditional or loop bodies in C++, use {}.
- In Python, use if … elif … else. In C++, use if…else if … else.
- Conditions in C++ must be enclosed within parentheses ().

Difference between python and C++: II

- In C++, a string is always enclosed in double quotes and cannot be enclosed in single quotes, unlike in Python. For example,

```
string greeting = "Hello, world"; //correct in C++
string greeting2 = 'Hello, world'; //wrong in C++
char ch = 'H'; //correct in C++
char ch2 = "H"; //wrong in C++
```

- In Python, incorrect indentation leads to syntax errors, whereas in C++, it doesn't cause an error but proper indentation is crucial for readability.

The First Program

```
#include <iostream> //ch01/hello.cpp
using namespace std;
int main() {
  cout << "Hello, World!" << endl;
  return 0;
}
```

Include iostream library for input and output

```
#include <iostream>
using namespace std;
int main() {
  cout << "Hello, World!" << endl;
  return 0;
}
```

using namespace std

- Use standard jargons of C++

```
#include <iostream>
using namespace std;
int main() {
  cout << "Hello, World!" << endl;
  return 0;
}
```

- Namespace is similar to field.
- In computer science (CS), WSL stands for Windows Subsystem for Linux. CS::WSL
- In sports, WSL refers to World Surf League. Sports::WSL

namespace is similar to field

- biscuit in US (image link)

US::biscuit

- biscuit in British (image link)

British::biscuit

In the absence of "using namespace std;"

Every occurrence of cout, cin, endl and string, needs to add std::

```
#include <iostream>
int main() {
  std::cout << "Hello, World!" << std::endl;
  return 0;
}
//some people think "using namespace std;"
//bad practice. See link.
```

Function main

- Every C++ program must contain a single `main` function, which serves as the entry point.
- All function names, including `main`, are followed by parentheses. In `main`'s case, the parentheses are empty.
- Additionally, braces {} are required to enclose all the code within the `main` function.

```
#include <iostream>
using namespace std;
int main() {
  …
}
```

- A project is like a company.
- Function main is like its CEO.
- All TO-DO lists are put in function main().

Output using cout statement

- To display content on the screen in C++, send them to the **cout** entity using the << operator, also known as the insertion operator.
- **endl** signifies end-of-line, causing the output to move to the next line.

```
#include <iostream>
using namespace std;
int main() {
  cout << "Hello, World!" << endl;
  return 0;
}
```

Use cout to print more than one output

- Display more than one thing by chaining or "streaming" multiple copies of the **<<** operator into the same statement:

```
cout << "A big " << "Hello, World!" << endl;
```

```
#include <iostream>
using namespace std;
int main() {
  cout << "Hello, World!" << endl;
  return 0;
}
```

return statement

- The main function returns an integer (int), which acts like an error code returned to the caller of the main function. Statement return 0; means the program runs successfully.

```
#include <iostream>
using namespace std;
int main() {
   cout << "Hello, World!" << endl;
   return 0;
}
```

- **"Hello World!"** is called a *string literal*.
- Put double-quotes, not single quotes, around a string.

Each statement in C++ ends with a semicolon ;

```
#include <iostream> //no ; after #include …
using namespace std;
int main() {
  cout << "Hello, World!" << endl;
  return 0;
}
```

Input and output

- Output to screen using cout , input from keyboard using cin.

```
#include <iostream>
using namespace std;
int main() {
    cout << "Enter an int: ";
    int num;
    cin >> num;
    cout << "You entered " << num << endl;
    return 0;
}
```

Warning: do not mix << and >>

cout << things_to_output; //things_to_output can be either a const like string literal or a variable.

//The << operator is used to push or insert the content into cout, the standard output device, which is typically the monitor.

cin >> variable; //A variable, once properly declared, can receive input using the >> operator, which pulls data from cin, the standard input device, such as a keyboard or console.

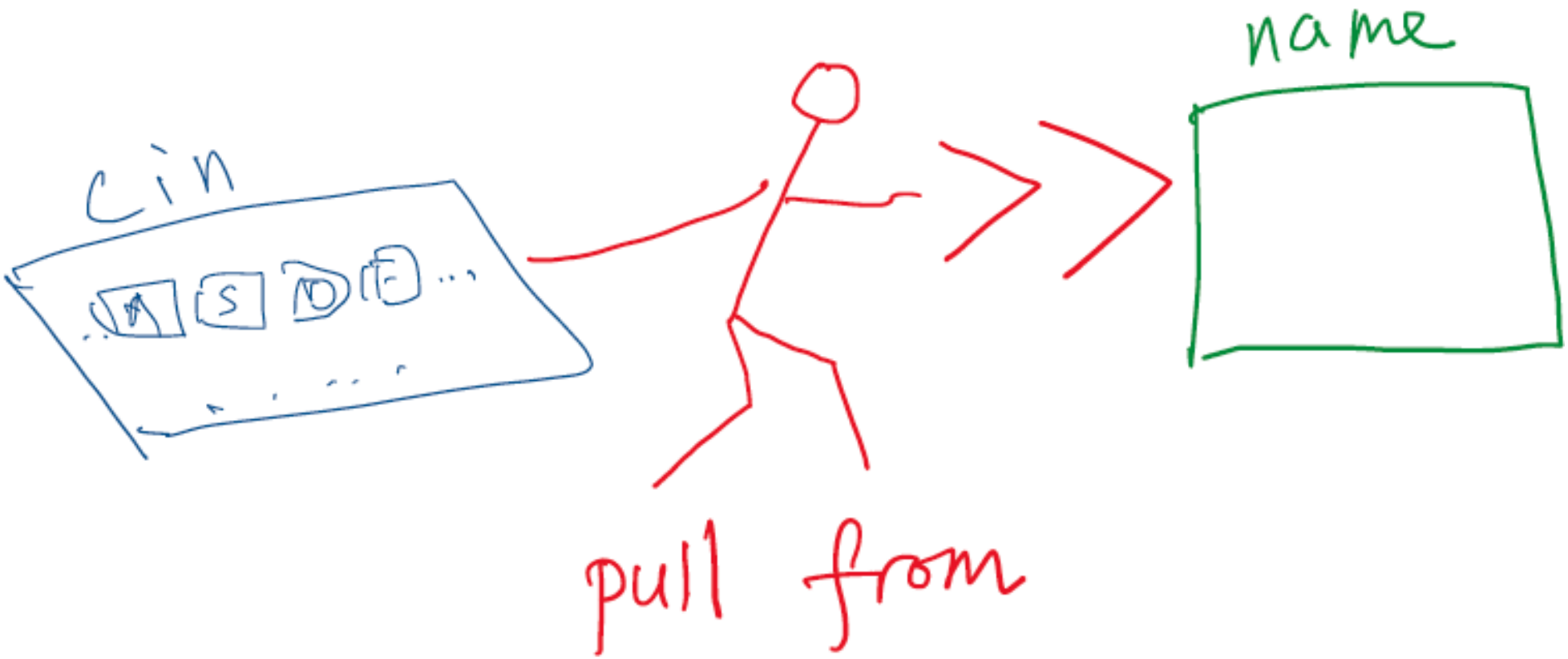
insertion (aka push) operator <<

- cout << variable; or cout << const;

<< is insertion / push operator

>> extraction (aka pull from) operator

- cin >> variable; //variable is declared before this statement.



Declare a variable

- Specify the type and name of the variable.

```
int i;
    //i is an integer variable
    //and can hold an integer.
    //int is the type, implying size, and
    //i is the name.
```

Declare an integer variable & initialize

- `int i;`

- `i = 5;`

- Declare and initialize (the first time assignment is also called initialization) a variable in two statements.

```
int i;
i = 5;
```

- Shortcut: declare and initialize in one statement.

```
int i = 5;
```

- Only one option can be chosen, since a variable can be declared once and only once.

Rules to name a variable

- Variable names must start with a letter or the underscore (_) character, and the remaining characters must be letters, numbers, or underscores.
- Variable names are **case-sensitive**, that is, can_volume and can_Volume are different names. For that reason, it is a good idea to use only lowercase letters in variable names.
- You cannot use reserved words such as new, double or return as names; these words are reserved exclusively for their special C++ meanings. See Appendix B.

primitive types of C++ used in this course

- Integer, ie, whole number: **int, long**
- floating-point number, ie, number with decimal parts: **float, double**
- character: **char**
- boolean, ie, true or false: **bool**
- Normally, if it is a whole number, use **int**, if the number might have decimal parts, use **double**.

Is string a primitive type in C++?

- NO.
- string is a class in C++.
- Unlike a primitive type, which has only data.
- A class is an encapsulation of data and operations on those data.

string is a class

- string (all small letters) variable stores a sequence of characters. Here is a string with values “Hello”.

index	0	1	2	3	4
Letter at index	‘H’	‘e’	‘l’	‘l’	‘o’

- Initialization
- 1st: primitive type style initialization,

```
string greeting = "Hello";
```

- 2nd: object instantiation in class

```
string greeting("Hello");
```

- A frog is an amphibian living in water or land.
- A string is initialized as a primitive type or a class.

Operations for String

- Data of string greeting: sequence of characters ‘H’, ‘e’, ‘l’, ‘l’, ‘o’

index	0	1	2	3	4
Letter at index	‘H’	‘e’	‘l’	‘l’	‘o’

- Operation for string greeting
  - length: number of characters
  - size: number of characters
  - at: characters at ith index, where the first character is indexed at 0
  - More will be covered in Chapter 3.

Assignment operator =

- Assignment operator = runs from right to left
  - First, evaluate the right hand side expression to get its value.
  - Put its value to the left hand side variable.
  - The value of the left hand side variable is changed.

```
int count = 1;
count = count + 1; //same as count++; and count += 1;
//Warning: count++; is a valid statement, but not count++2;
```

Assignment Operator = Illustration

- Animal pet = ; //image link
- Animal pet2 = ; //image link

Link to answer: https://forms.gle/dx7aQkM4FqnghKfT7

After pet = pet2; what are the values of pet and pet2?

(A) pet is cat, pet2 is dog

(B) pet is dog, pet2 is dog

(C) pet is cat, pet2 is cat

(D) pet is dog, pet2 is cat

Arithmetic operator

- Add +, minus -, multiply *, divide /, modulo (aka remainder) %
- When both numerator or denominate are integers, the result is the quotient with decimal parts truncated.
  - For example, 1 / 2 returns 0. However, 1.0 / 2 returns 0.5.
- Modulo or reminder operator %.
  - To calculate m % n, where both m and n are integers, imagine divide m pens among n persons, and each person should get the same number of pens, **how many pens are left**.
  - For example, 5 % 3 returns 2. What is 6 % 3?

Exercise on input/output statements and arithmetic operation

- Input your name as a string.
- Enter your current age as an int.
- Subtract the age by 5, print out your age five years ago.
- Sample input/output:

Enter your name: YI

Enter your age: 95

Your age in 5-year ago is 90

condition

- Condition is an expression that returns true or false
- Conditions in C++ are enclosed in a pair of parentheses ().
- Comparison operator are as follows.
  - ❖The == operator checks if the left expression is equal to the right expression. It returns true if they are equal and false otherwise.
  - ❖The != operator checks if the left expression is not equal to the right expression. It returns true if they are not equal, and false if they are equal.

Equality Comparison Operator == Illustration

- Animal pet = ; //image link
- Animal pet2 = ; //image link

Link to enter an answer: https://forms.gle/UHQepv5aE1xj7G6r6

After (pet == pet2) what are the values of pet and pet2?

(A) pet is cat, pet2 is dog

(B) pet is dog, pet2 is dog

(C) pet is cat, pet2 is cat

(D) pet is dog, pet2 is cat

Condition: II

- Other comparison operators are as follows.
  - ❖less than operator <

    if the left expression is less than the right expression, return true, otherwise, return false.
  - ❖Less than or equal operator <=

    if the left expression is less than or equal to the right expression, return true, otherwise, return false. **Warning**: no space between < and = and the order cannot be switched.
  - ❖larger than operator >
  - ❖larger than or equal to operator >=

composite condition

- Use && (and) or || (or) to connect single conditions.
- Suppose a, b, c are integers properly declared and initialized. Write a condition that a is larger than or equal to b and a is larger than equal to c.
- Write condition that variable score, properly declared and initialized, is in the range [0, 100], where both ends are included.
  - **Warning:** cannot write 0 <= score <= 100
- Write condition that score is out of range of [0, 100].

Exercise: write conditions

Suppose variable score are properly declared and initialized.

- Write conditions that score is in the range of [0, 100].

(A) (score > 0 && score < 100)

(B) (score >= 0 && score <= 100)

(C) (score > 0 || score < 100)

(D) (score >= 0 || score <= 100)

(E) (! (score < 0 || score > 100))

(F) (0 <= score <= 100)

Exercise II: write conditions

Suppose variable score are properly declared and initialized.

- Write conditions that score is OUT of the range of [0, 100].

(A) (score < 0 || score > 100)

(B) (score <= 0 || score >= 100)

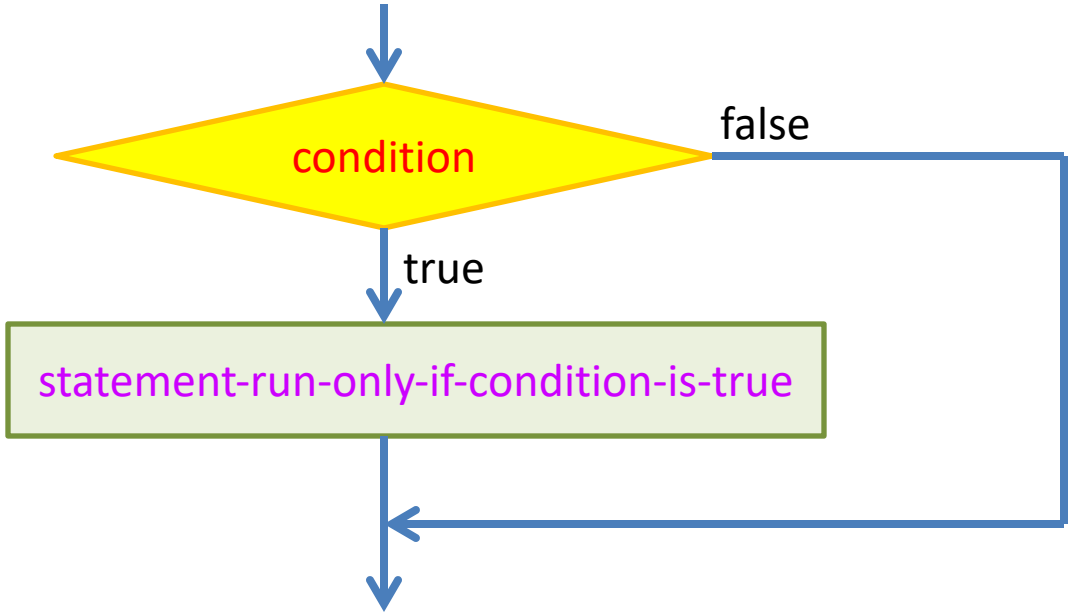
(C) (! (score >= 0 && score <= 100))

(D) (score < 0 && score > 100)

(E) (score <= 0 && score >= 100)

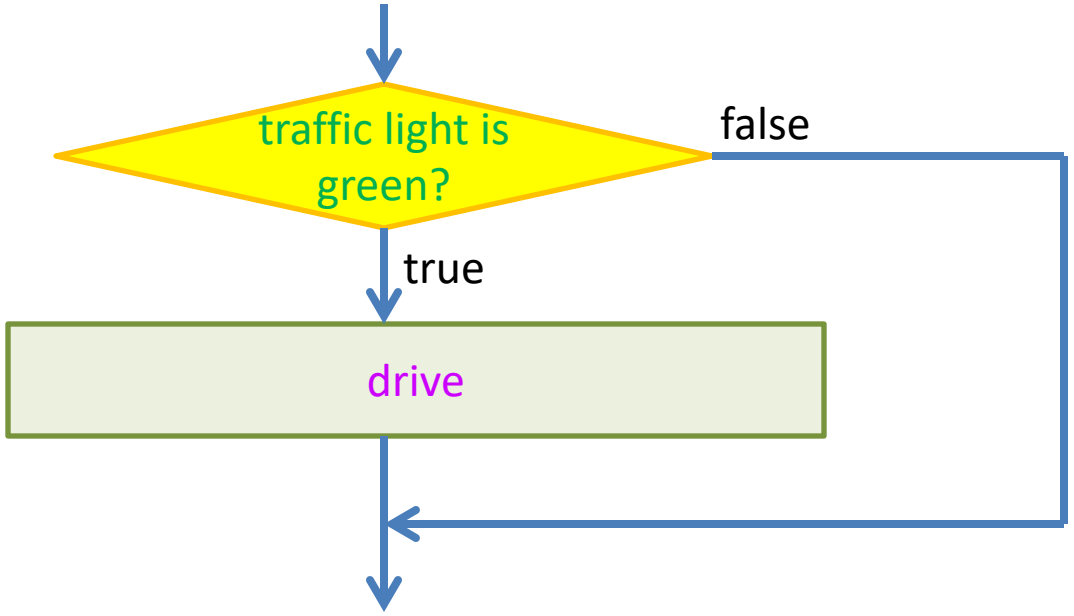
if-statement

```
if (condition)
    statement-run-only-if-condition-is-true;
```



if-statement example

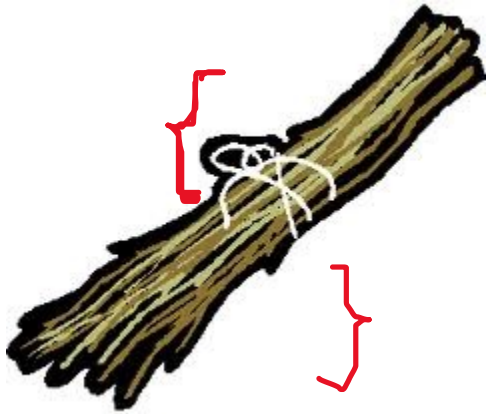
```
if (traffic_light == “green”)
    print “drive”
```



Bundle statements with { and }

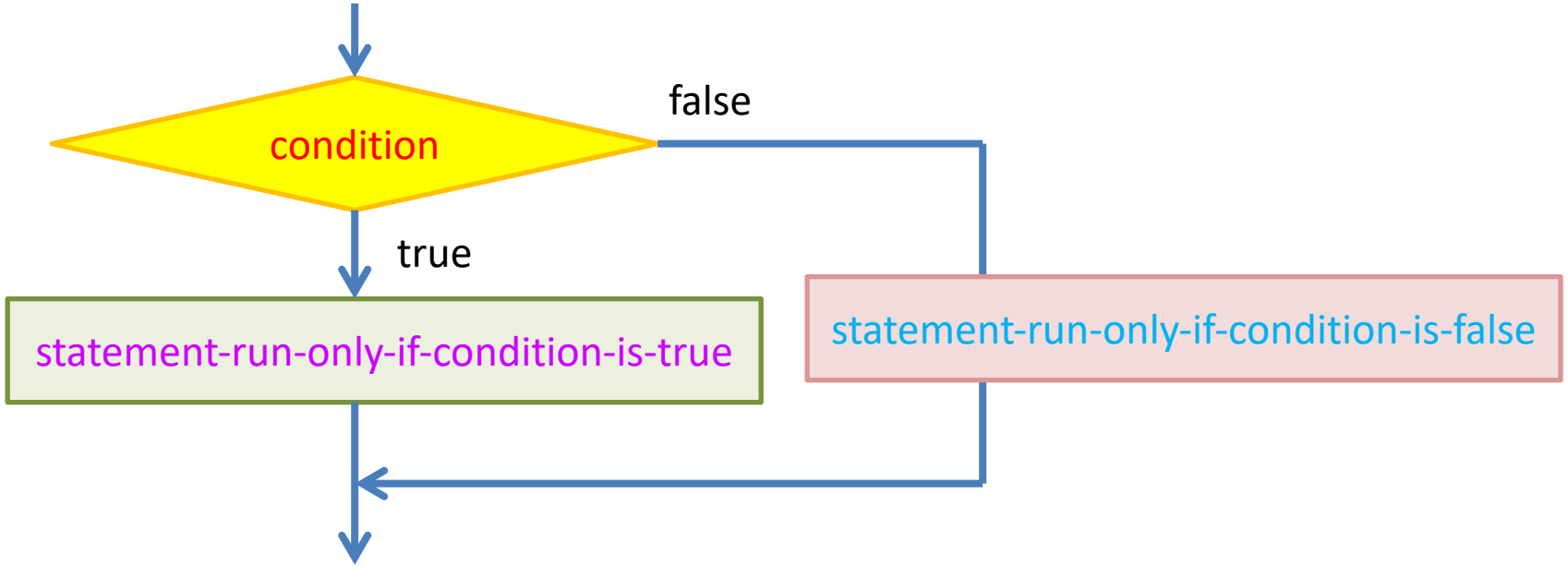
Run more than one statement if something happens:

```
if (hurricane hits)
{
    evacuate people in low-laying area;
    close down schools;
}
```



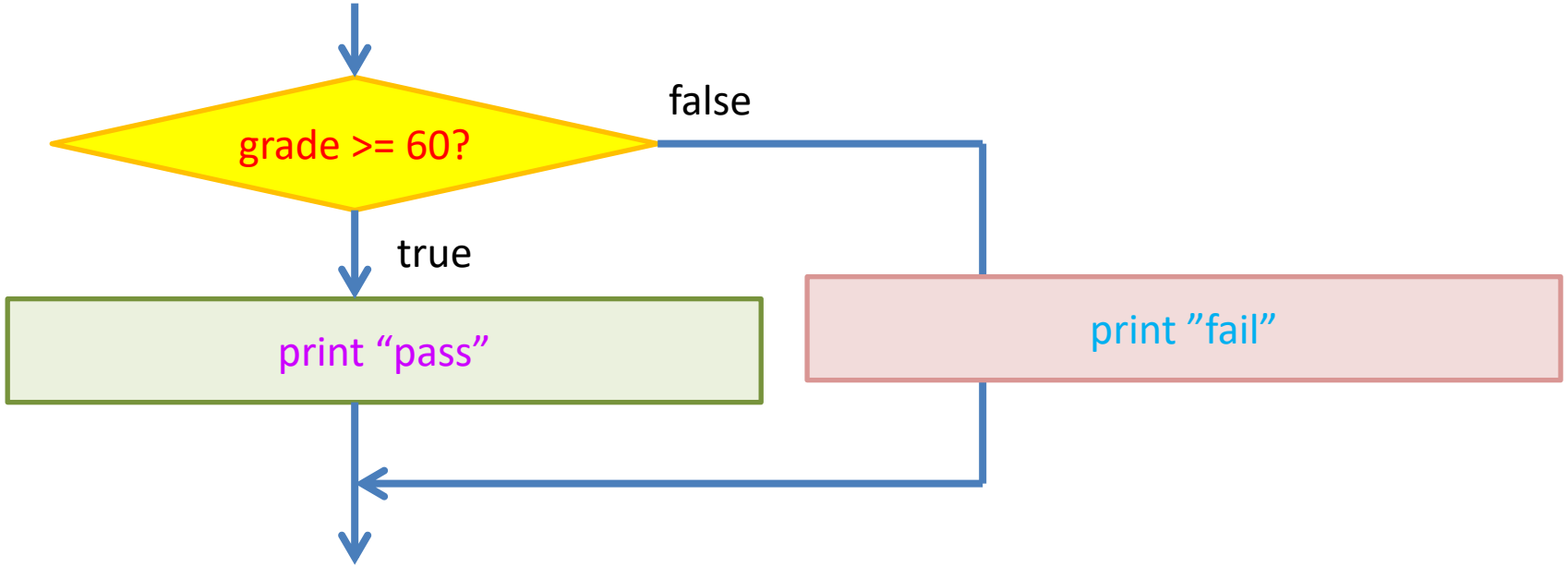
if-else statement

```
if (condition)
    statement-run-only-if-condition-is-true;
else statement-run-only-if-condition-is-false;
```



if-else statement example

```
if (condition)
    statement-run-only-if-condition-is-true;
else statement-run-only-if-condition-is-false;
```



There might be more than two possibilities …

- In an exam, we may have only two outcomes (pass or fail).
- Sometimes, life has more than two possibilities. For example,
  - Signals of a traffic light
  - Even an exam can have A, B, C, D, F grades.
  - Tax brackets for different household incomes

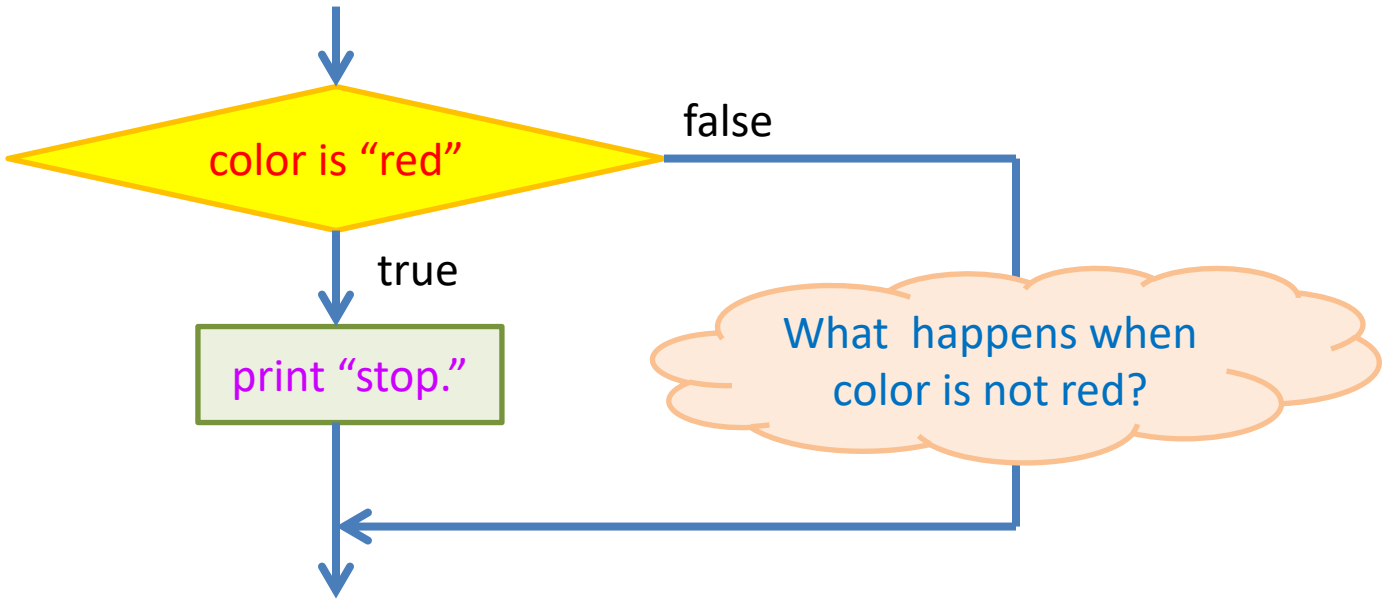
Nested if-else: traffic light

Check the state of traffic light, if it is green, then print out “go!”; if it is red, then print out “stop.”; if it is yellow, then print out “slow down.”.

Print “stop.”	if color is red
Print “slow down.”	if color is yellow
Print “go!”	if color is green

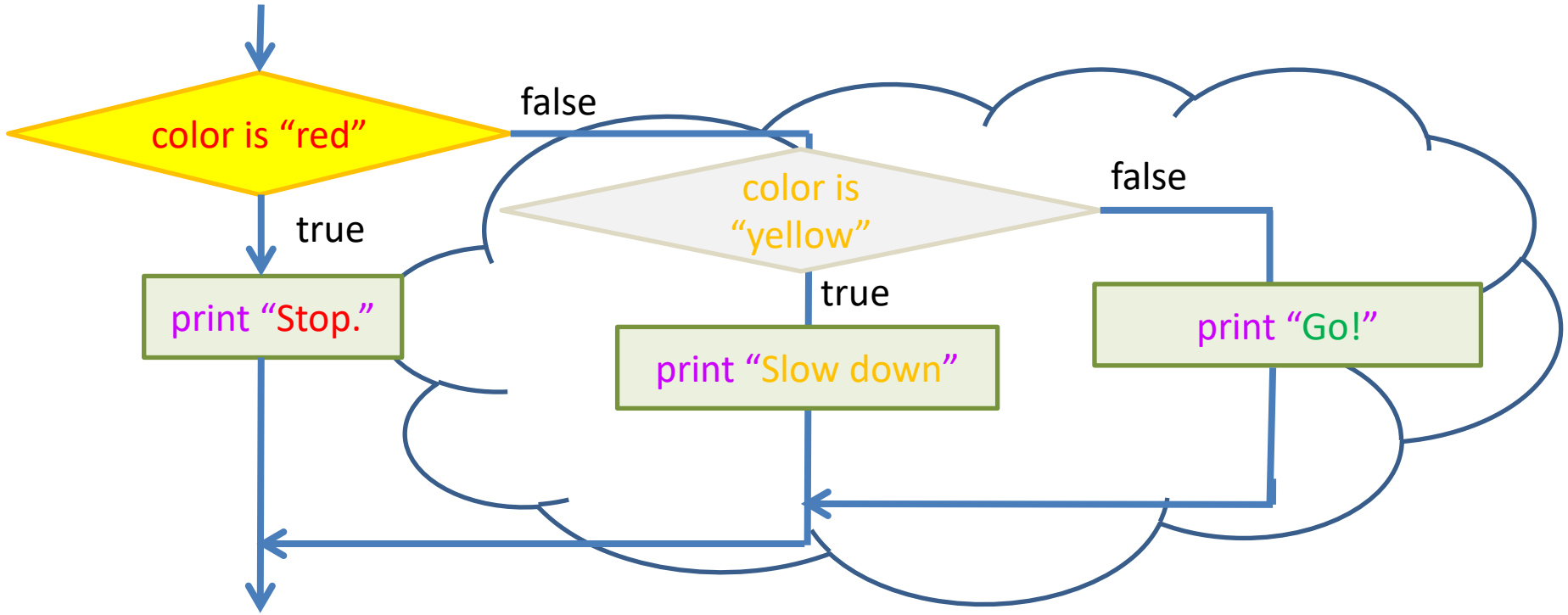
Nest if-else traffic light

A traffic light has three possible colors: red, green, and yellow.



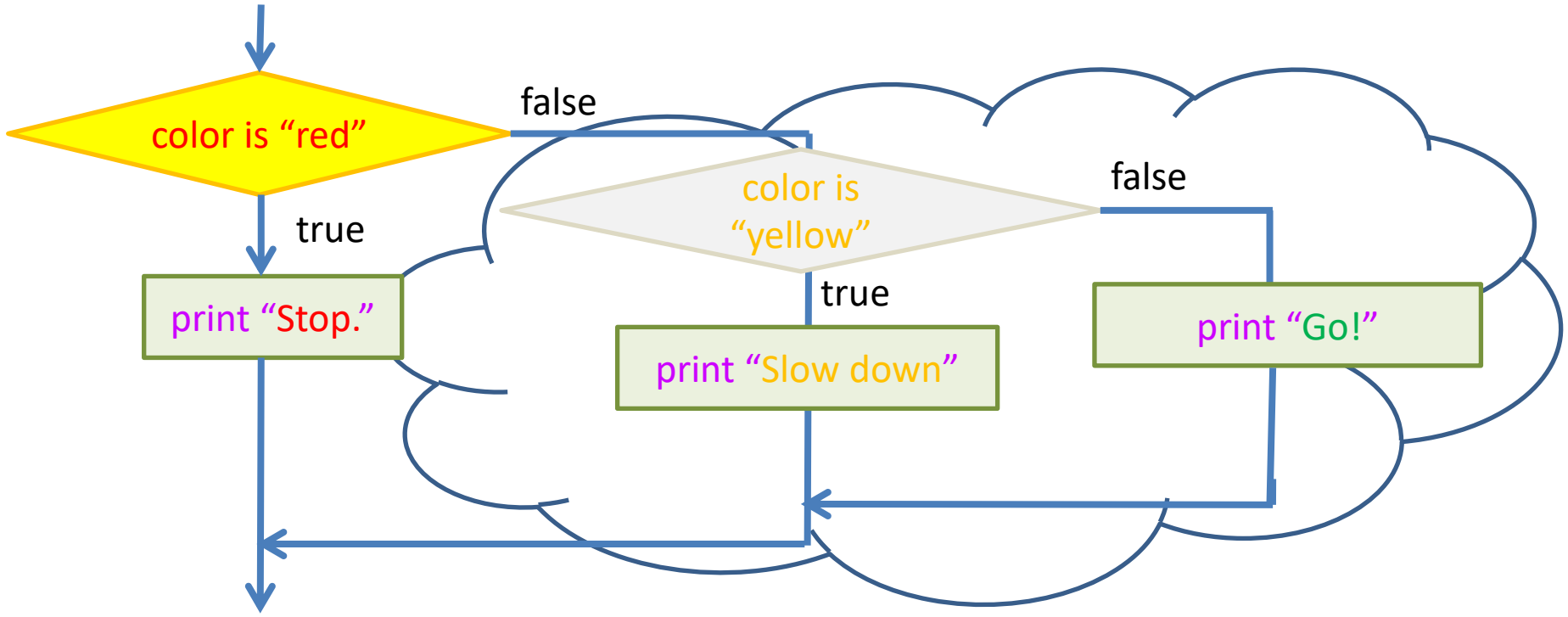
Nest if-else traffic light: II

A traffic light has three possible colors: red, green, and yellow.



Nest if-else traffic light: III

Translate the following chart into code (Hint: cloud part first. What kind statement is it?)



Nested if-else

Check the state of traffic light, if it is green, then print out “go!”; if it is red, then print out “stop.”; if it is yellow, then print out “slow down.”.

Print “stop.”	if color is red
Print “slow down.”	if color is yellow
Print “go!”	if color is green

Nested if-else: II

Check the state of traffic light, if it is green, then print out "go!"; if it is red, then print out "stop."; if it is yellow, then print out "slow down.".

If color is red, print "stop."

Otherwise (the color is not red, it can be yellow or green)

if color is yellow, then print "slow down."

Otherwise (the color is neither red nor yellow, it must be green)

Print "go!"

Nested if-else

Check the state of traffic light, if it is green, then print out “go!”; if it is red, then print out “stop.”; if it is yellow, then print out “slow down.”

Print “stop.” if color is red

Print “slow down.” if color is yellow

Print “go!” if color is green

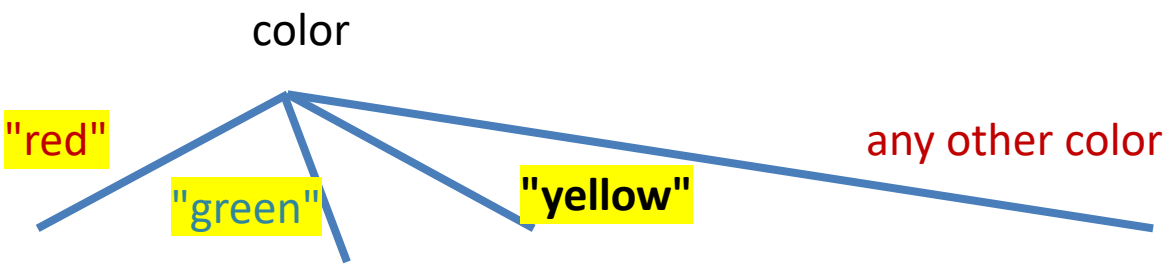
```
if (color=="red")
    … //print out "stop."
else if (color == "yellow")
        … //print out "slow down."
    else … //print out "go!"
```

equality operator == vs assignment operator =

- == operator checks whether the left hand side expression equals to the right hand side expression or not. No change to expressions.
- Do not confuse == (equality operator) with = (assignment operator).
- The assignment operator = operates from right to left.
  - It first evaluates the expression on the right-hand side and then assigns its value to the variable on the left-hand side.

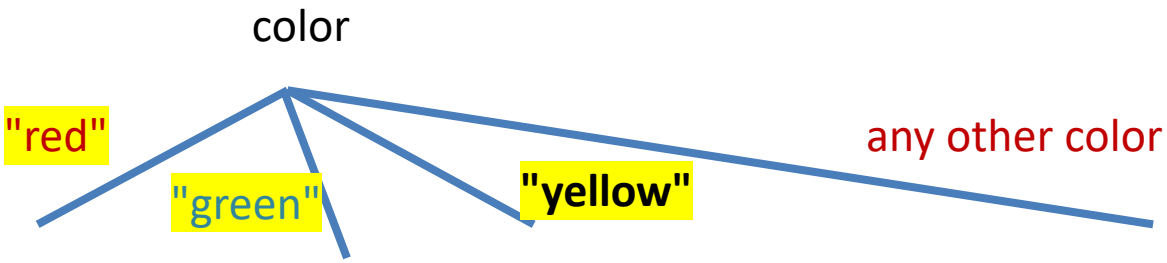
Code to check traffic light

```
#include <iostream>
#include <string>
using namespace std;
int main () {
    string color;
    cout << "Enter color: ";
    cin >> color;
```



Code to check traffic light

```
    if (color == "red")
      cout << "stop" << endl;
    else if (color == "green")
            cout << "go" << endl;
    else if (color == "yellow")
            cout << "slow down" << endl;
    else cout << "wrong color" << endl;
    return 0;
}
```



- Warning: do **not** add condition (…) right after else.
- Condition (…) should follow keyword if.

Nested if-else statement

Without {}, else is bounded with the closest if.

```
if (a >= b)
    if (a >= c) //what can we say for a, b, c?
        …
    else //what can we say for a, b, c?
        …
```

Nested if-else statement

With {}, else is bounded with the closest if not in {}.

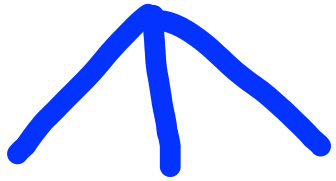
```
if (a >= b)
{
  if (a >= c)
      ...
}
else
    ...
```

Approach 1: Find the max of three inputs using nested if-else statement

```
//TODO: declare a, b, c are ints and initialize them.
int maxValue;
if (a >= b) { //can b be a maximum when a >= b?
  if (…) //which elements should be compared to find out the max'm?
    maxValue = …;
   else //a >= b and a < c
        maxValue = …;
}
else { //a < b TODO: work on the rest
}
//TODO: print maxValue
```

Approach 2: Find the max of three inputs using composite condition and nested if-else statement

find max of a, b, c



a is max b is max c is max

a is max when…

b is max when …

otherwise

otherwise
c is max

```
//TODO: declare a, b, c are ints and initialize them.
int maxValue;
if (                                )
    maxValue = a;
else //a is NOT the max, compare b and c
    if (…)
        maxValue = b;
    else
        maxValue = …;
//TODO: print maxValue
```

Better way to find the maximum of three integers

- Finding the maximum of three integers by comparing different pairs of integers works, but the approach is not scalable, for example, find the maximum of 4 integers.
- It is like to a queue of three kids competing to be a model.
  - The first child has no competitor, so he/she is the chosen model.
  - The second child comes and competes with the chosen model, if the new comer is cuter, then he/she becomes the new model.
  - The third child comes and competes with the chosen model, if the new comer is cuter, then he/she becomes the new model.

Find a winner model in 3 babies

Who may be the winner model?

Show Desktop.scf

Find a winner model in 3 babies

Rules to select the winner model:

- Babies may join competition at any time.
- If the incoming baby is cuter than the current winner model, choose the new baby as model; otherwise, keep the current model.

Find a winner model in 3 babies

In January, Baby Alex came and was the winner model. (no competitor yet, oh yah!!)

Find a winner model in 3 babies

After several months, Bob comes.

Bob competes with

winner model .

If Bob is cuter, then he

becomes winner model .

winner model

Find a winner model in 3 babies

After several months, Chris comes.

Chris competes with winner model .

If Chris is cuter, then he becomes winner model .

winner model

Find a winner model in 3 babies

How to choose a winner model?

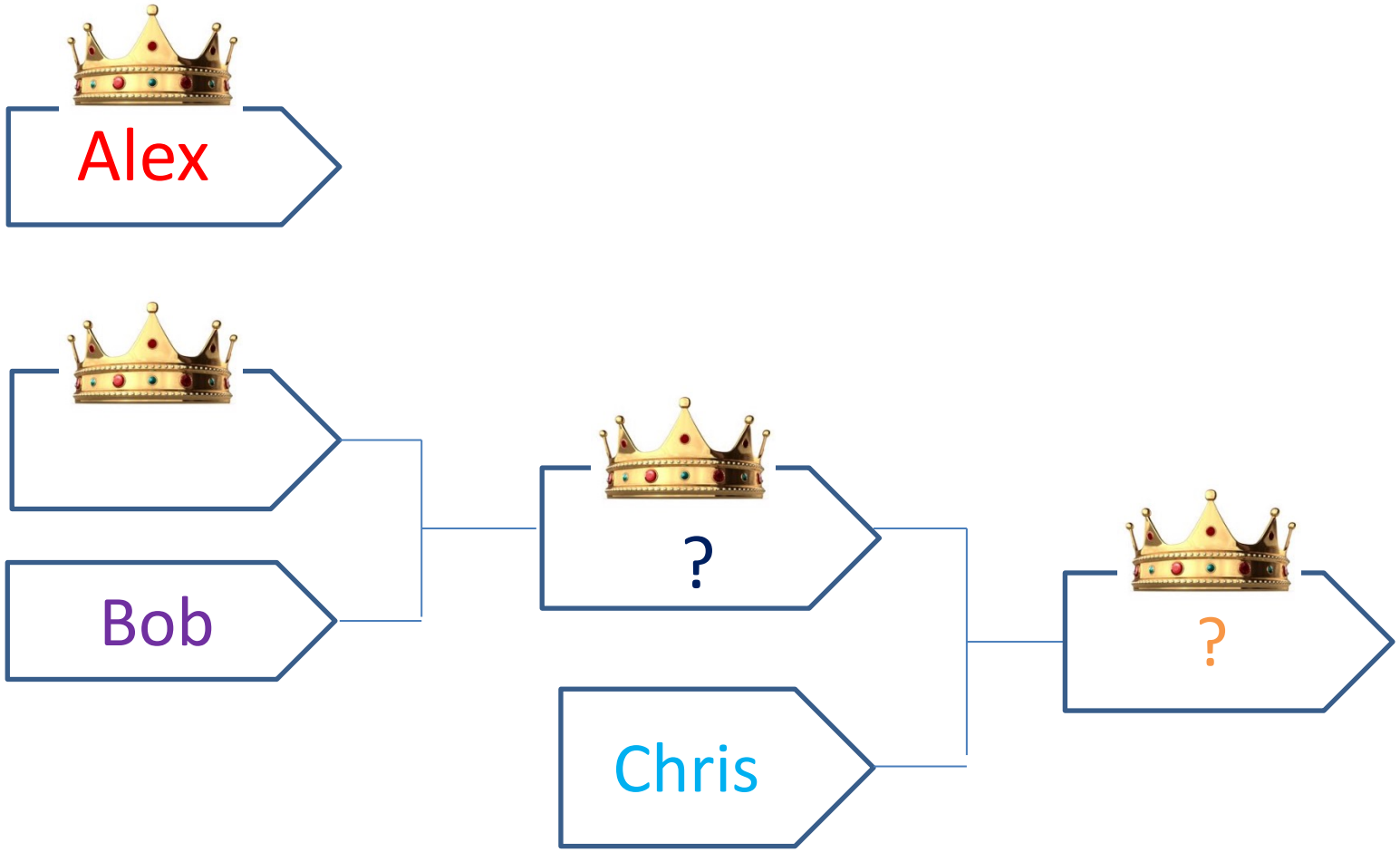
(1) The first comer is awarded as the winner model automatically.

(2) Every time a new comer comes, he/she challenges the current winner model. If the new comer wins, he/she becomes the new winner model (what if the challenger does not win?)

(3) Repeat (2) for every new comer.

Find a winner model in 3 babies

Hint: tournament (one loss & immediate out)



Even or Odd

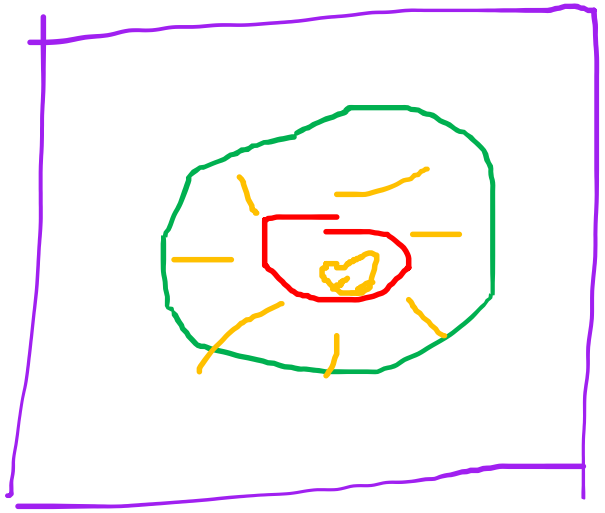
If an int is divided by 2, then it is even, otherwise, it is odd.

Leap Year

- In the **modern Gregorian calendar**, a year is a *leap year* if it is divisible by 4, but century years are not leap years unless they are divisible by 400.
- That is, a leap year is one of the following
  - Divisible by 4 but not by 100

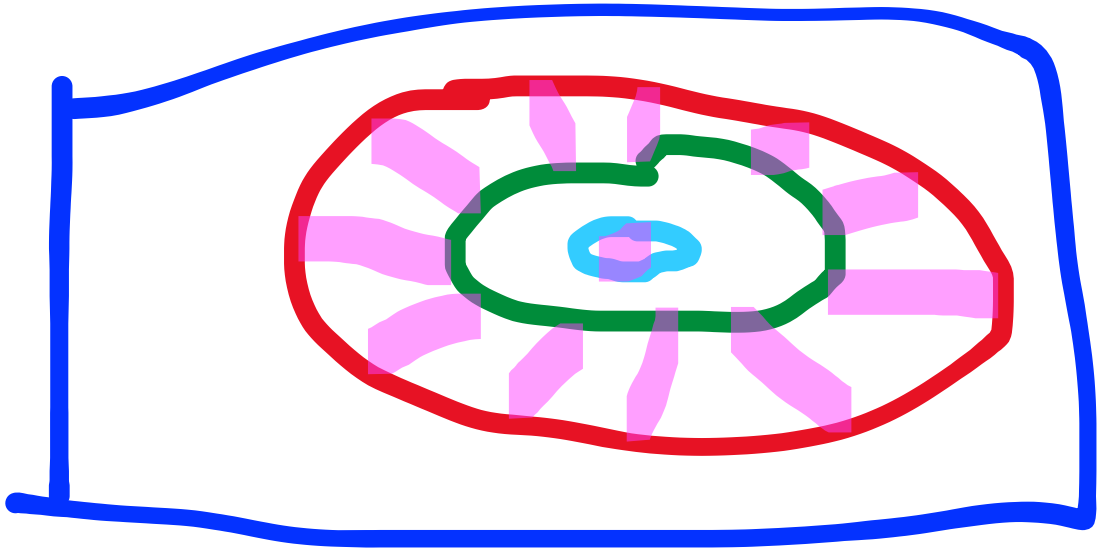
  OR

  - Divisible by 400



Leap Year

- In the **modern Gregorian calendar**, a year is a *leap year* if it is divisible by 4, but century years are not leap years unless they are divisible by 400.

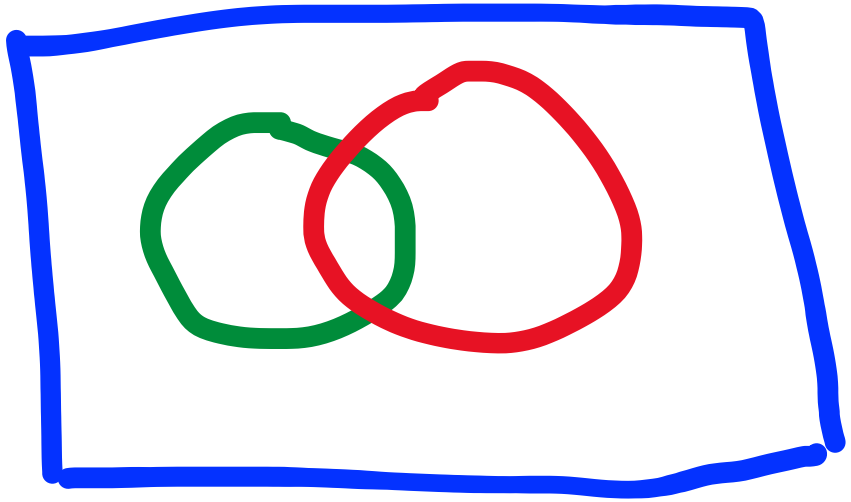


- Blue box means all years
- Red circle means year divisible by 4.
- Green circle means years divisible by 100.
- Cyan circle means year divisible by 400.
- Shaded areas means leap year.

Divisible by both 2 and 5, divisible by 2 or 5 only, divisible by neither 2 nor 5

Write a C++ main function that takes an integer from console, then report that it can be

- divided by 2 only,
- divided by 5 only,
- divided by both 2 and 5, or
- divided by neither 2 nor 5.



- Blue box represents all integers
- Green circle means all ints divided by 2
- Red circle means all ints divided by 5

Summary

- A problem normally involves three steps.
  - Input
  - Process input and get an output
  - Print output